

# ANGULAR HANDS-ON

## Project Theme: Student Course Portal

### Executive Summary & Architecture Overview

The Student Course Portal is a single, unified Angular application designed to handle student course management, enrollment workflows, profile tracking, state persistence, backend API interactions, and automated testing.

### Project Directory Structure

```
Angular_HandsOn/  
├── <YourName>/  
│   └── student-course-portal/  
│       ├── db.json  
│       ├── notes.txt  
│       ├── angular.json  
│       ├── package.json  
│       └── src/  
│           ├── main.ts  
│           └── app/  
│               ├── app.routes.ts  
│               ├── app.config.ts  
│               ├── app.component.ts  
│               ├── models/  
│               │   └── course.model.ts  
│               ├── services/  
│               │   ├── course.service.ts  
│               │   ├── enrollment.service.ts  
│               │   ├── loading.service.ts  
│               │   └── auth.service.ts  
│               ├── interceptors/  
│               │   ├── auth.interceptor.ts  
│               │   ├── error-handler.interceptor.ts  
│               │   └── loading.interceptor.ts  
│               ├── guards/  
│               │   ├── auth.guard.ts  
│               │   └── unsaved-changes.guard.ts  
│               ├── store/  
│               │   └── course/  
│               │       ├── course.actions.ts  
│               │       ├── course.reducer.ts  
│               │       ├── course.selectors.ts  
│               │       └── course.effects.ts  
│               ├── components/  
│               │   ├── header/  
│               │   └── course-card/  
│               ├── pages/  
│               │   ├── home/  
│               │   ├── course-list/  
│               │   ├── course-detail/  
│               │   ├── student-profile/  
│               │   ├── not-found/  
│               │   └── enrollment-form/  
│               ├── features/  
│               │   └── enrollment/  
│               ├── directives/  
│               │   └── highlight.directive.ts  
│               ├── pipes/  
│               │   └── credit-label.pipe.ts
```

# Hands-On 1: Environment Setup, Project Structure & First Component

## Implementation Details

- **Scaffolded Project:** Initialized project using Angular CLI v20 with standard routing and modern standalone architecture (app.config.ts and app.routes.ts).
- **Core Components:** Configured core visual page components: HeaderComponent, HomeComponent, CourseListComponent, and StudentProfileComponent.
- **Documentation:** Documented core project settings and budget parameters in notes.txt.

## Key Code Artifacts

### notes.txt

```
// notes.txt
angular.json: Main CLI configuration file defining project builds, targets, assets, and asset bundle limits.
tsconfig.json: Master TypeScript compiler configuration for the workspace.
tsconfig.app.json: TypeScript compiler options specifically tailored for compiling the application code.
package.json: Lists project dependencies, scripts, devDependencies, and package version specifications.
src/main.ts: Application bootstrapping entry point launching the root component/config.
src/app/app.config.ts: Global application providers configuration (Standalone API approach).
src/app/app.component.ts: Root application component hosting top-level templates and routing outlets.
src/index.html: The host single-page HTML skeleton hosting the root component (<app-root>).
```

## Header Component TS & HTML

```
// src/app/components/header/header.component.ts
import { Component } from '@angular/core';
import { RouterLink } from '@angular/router';

@Component({
  selector: 'app-header',
  standalone: true,
  imports: [RouterLink],
  templateUrl: './header.component.html',
  styleUrls: ['./header.component.css']
})
export class HeaderComponent {}
```

```
// src/app/components/header/header.component.html
<nav class="navbar">
  <div class="brand">Student Course Portal</div>
  <ul class="nav-links">
    <li><a routerLink="/">Home</a></li>
    <li><a routerLink="/courses">Courses</a></li>
    <li><a routerLink="/enroll">Enroll</a></li>
    <li><a routerLink="/profile">Profile</a></li>
  </ul>
</nav>
```

## Home Component TS & HTML

```
// src/app/pages/home/home.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  standalone: true,
  imports: [],
```

```
templateUrl: './home.component.html',
styleUrls: ['./home.component.css']
})
export class HomeComponent {}
```

```
// src/app/pages/home/home.component.html
<div class="home-container">
  <h1>Welcome to the Student Course Portal</h1>
  <p>Manage your academic journey, browse courses, and monitor enrollment status easily.</p>

  <div class="stats-row">
    <div class="stat-card">Courses Available: <strong>12</strong></div>
    <div class="stat-card">Enrolled: <strong>3</strong></div>
    <div class="stat-card">GPA: <strong>3.8</strong></div>
  </div>
</div>
```

## Hands-On 2: Data Binding, Lifecycle Hooks & Component Communication

### Implementation Details

- **Binding Types:** Implemented four dynamic Angular binding types (Interpolation, Property, Event, Two-Way via FormsModule).
- **Lifecycle Hooks:** Managed component lifecycles using `ngOnInit`, `ngOnChanges`, and `ngOnDestroy`.
- **Parent-Child Communication:** Created robust communication patterns between `CourseListComponent` and `CourseCardComponent` using `@Input` and `@Output` decorated observables.

### Key Code Artifacts

```
// src/app/pages/home/home.component.ts (Binding & Hooks Example)
import { Component, OnInit, OnDestroy } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-home',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.css']
})
export class HomeComponent implements OnInit, OnDestroy {
  portalName: string = 'Student Course Portal';
  isPortalActive: boolean = true;
  message: string = '';
  searchTerm: string = '';

  ngOnInit(): void {
    console.log('HomeComponent initialised courses loaded');
  }

  ngOnDestroy(): void {
    console.log('HomeComponent destroyed');
  }

  onEnrollClick(): void {
    this.message = 'Enrollment opened!';
  }
}
```

```
// src/app/components/course-card/course-card.component.ts
import { Component, Input, Output, EventEmitter, OnChanges, SimpleChanges } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Course } from '../../models/course.model';

@Component({
  selector: 'app-course-card',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './course-card.component.html',
  styleUrls: ['./course-card.component.css']
})
export class CourseCardComponent implements OnChanges {
  @Input() course!: Course;
  @Output() enrollRequested = new EventEmitter<number>();

  ngOnChanges(changes: SimpleChanges): void {
    if (changes['course']) {
      console.log('Course input changed:',
        changes['course'].previousValue,
        '->',
        changes['course'].currentValue
      );
    }
  }

  onEnroll(): void {
    this.enrollRequested.emit(this.course.id);
  }
}
```

## Hands-On 3: Directives & Pipes — Built-in and Custom

### Implementation Details

- **Directives:** Applied structural directives (\*ngIf, \*ngFor with performance-optimized trackBy, \*ngSwitch) and attribute directives (ngClass, ngStyle).
- **Custom Enhancements:** Created custom attribute directive HighlightDirective and custom Pipe CreditLabelPipe.

### Key Code Artifacts

```
// src/app/directives/highlight.directive.ts
import { Directive, ElementRef, HostListener, Input, Renderer2 } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  standalone: true
})
export class HighlightDirective {
  @Input() appHighlight: string = 'yellow';

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  @HostListener('mouseenter') onMouseEnter(): void {
    this.renderer.setStyle(this.el.nativeElement, 'backgroundColor', this.appHighlight);
  }

  @HostListener('mouseleave') onMouseLeave(): void {
    this.renderer.removeStyle(this.el.nativeElement, 'backgroundColor');
  }
}
```

```
}  
}
```

```
// src/app/pipes/credit-label.pipe.ts  
import { Pipe, PipeTransform } from '@angular/core';  
  
@Pipe({  
  name: 'creditLabel',  
  standalone: true  
})  
export class CreditLabelPipe implements PipeTransform {  
  transform(value: number | null | undefined): string {  
    if (value === null || value === undefined || value === 0) {  
      return 'No Credits';  
    }  
    return value === 1 ? '1 Credit' : `${value} Credits`;  
  }  
}
```

## Hands-On 4: Template-Driven Forms & Validation

### Implementation Details

- **Form Setup:** Constructed an interactive Enrollment Form with full client-side HTML structural validations (required, minlength, email).
- **Validation Feedback:** Exposed error reporting based on control interaction flags (#nameCtrl='ngModel').

### Key Code Artifacts

```
// src/app/pages/enrollment-form/enrollment-form.component.ts  
import { Component } from '@angular/core';  
import { CommonModule } from '@angular/common';  
import { FormsModule, NgForm } from '@angular/forms';  
  
@Component({  
  selector: 'app-enrollment-form',  
  standalone: true,  
  imports: [CommonModule, FormsModule],  
  templateUrl: './enrollment-form.component.html',  
  styleUrls: ['./enrollment-form.component.css']  
})  
export class EnrollmentFormComponent {  
  isSubmitted = false;  
  
  student = {  
    studentName: '',  
    studentEmail: '',  
    courseId: null,  
    preferredSemester: 'Odd',  
    agreeToTerms: false  
  };  
  
  onSubmit(form: NgForm): void {  
    if (form.valid) {  
      console.log('Form Submitted!', form.value);  
      this.isSubmitted = true;  
    }  
  }  
}
```

# Hands-On 5: Reactive Forms & Custom Validators

## Implementation Details

- **FormBuilder Architecture:** Built programmatic forms using FormBuilder, FormGroup, FormControl, and dynamic FormArray elements.
- **Custom Validation:** Added synchronous custom validation (noCourseCode) and async validation (simulateEmailCheck).

## Key Code Artifacts

```
// src/app/pages/reactive-enrollment-form/reactive-enrollment-form.component.ts
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ReactiveFormsModule, FormBuilder, FormGroup, FormArray, Validators, AbstractControl, ValidationErrors } from '@angular/forms';
import { Observable, of } from 'rxjs';
import { delay } from 'rxjs/operators';

// Custom Synchronous Validator
export function noCourseCode(control: AbstractControl): ValidationErrors | null {
  const val = String(control.value || "");
  return val.startsWith('XX') ? { noCourseCode: true } : null;
}

// Custom Asynchronous Validator
export function simulateEmailCheck(control: AbstractControl): Observable<ValidationErrors | null> {
  const isTaken = String(control.value).includes('test@');
  return of(isTaken ? { emailTaken: true } : null).pipe(delay(800));
}

@Component({
  selector: 'app-reactive-enrollment-form',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './reactive-enrollment-form.component.html'
})
export class ReactiveEnrollmentFormComponent implements OnInit {
  enrollForm!: FormGroup;

  constructor(private fb: FormBuilder) {}

  ngOnInit(): void {
    this.enrollForm = this.fb.group({
      studentName: ['', [Validators.required, Validators.minLength(3)]],
      studentEmail: ['', [Validators.required, Validators.email], [simulateEmailCheck]],
      courseId: [null, [Validators.required, noCourseCode]],
      preferredSemester: ['Odd', Validators.required],
      agreeToTerms: [false, Validators.requiredTrue],
      additionalCourses: this.fb.array([])
    });
  }

  get additionalCourses(): FormArray {
    return this.enrollForm.get('additionalCourses') as FormArray;
  }

  addCourse(): void {
    this.additionalCourses.push(this.fb.control("", Validators.required));
  }

  removeCourse(index: number): void {
    this.additionalCourses.removeAt(index);
  }
}
```

```

onSubmit(): void {
  if (this.enrollForm.valid) {
    console.log('Value:', this.enrollForm.value);
    console.log('Raw Value:', this.enrollForm.getRawValue());
  }
}
}
}

```

## Hands-On 6: Services & Dependency Injection

### Implementation Details

- **Centralized Services:** Created centralized services (CourseService, EnrollmentService) to manage shared domain state and operations.
- **Scope Demonstration:** Demonstrated root singletons vs. component-scoped providers.

### Key Code Artifacts

```

// src/app/services/course.service.ts
import { Injectable } from '@angular/core';
import { Course } from '../models/course.model';

@Injectable({ providedIn: 'root' })
export class CourseService {
  private courses: Course[] = [
    { id: 1, name: 'Angular Basics', code: 'CS101', credits: 3, gradeStatus: 'passed' },
    { id: 2, name: 'Advanced TypeScript', code: 'CS202', credits: 4, gradeStatus: 'pending' },
    { id: 3, name: '.NET Web API', code: 'CS303', credits: 4, gradeStatus: 'passed' }
  ];

  getCourses(): Course[] {
    return this.courses;
  }

  getCourseById(id: number): Course | undefined {
    return this.courses.find(c => c.id === id);
  }

  addCourse(course: Course): void {
    this.courses.push(course);
  }
}

```

## Hands-On 7: Angular Routing — Guards, Lazy Loading & Route Data

### Implementation Details

- **Routing Architecture:** Implemented nested routes, query parameters, lazy-loaded feature modules.
- **Navigation Protection:** Protected routes with CanActivate authentication guards and CanDeactivate unsaved changes prevention guards.

### Key Code Artifacts

```

// src/app/app.routes.ts
import { Routes } from '@angular/router';
import { HomeComponent } from './pages/home/home.component';
import { CourseListComponent } from './pages/course-list/course-list.component';

```

```

import { CourseDetailComponent } from './pages/course-detail/course-detail.component';
import { StudentProfileComponent } from './pages/student-profile/student-profile.component';
import { NotFoundComponent } from './pages/not-found/not-found.component';
import { AuthGuard } from './guards/auth.guard';
import { UnsavedChangesGuard } from './guards/unsaved-changes.guard';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'courses',
    children: [
      { path: '', component: CourseListComponent },
      { path: ':id', component: CourseDetailComponent }
    ]
  },
  { path: 'profile', component: StudentProfileComponent, canActivate: [AuthGuard] },
  {
    path: 'enroll',
    loadChildren: () => import('./features/enrollment/enrollment.routes').then(m => m.ENROLLMENT_ROUTES),
    canActivate: [UnsavedChangesGuard]
  },
  { path: '**', component: NotFoundComponent }
];

```

## Hands-On 8: HTTP Client — API Integration, Observables & Interceptors

### Implementation Details

- **API Operations:** Configured REST API operations connected to a local JSON Server (db.json).
- **Interceptors:** Built three interceptors (authInterceptor, errorHandlerInterceptor, loadingInterceptor).

### Key Code Artifacts

```

// db.json
{
  "courses": [
    { "id": 1, "name": "Angular Framework Essentials", "code": "ANG101", "credits": 3, "gradeStatus": "passed" },
    { "id": 2, "name": "Enterprise Architecture", "code": "CS401", "credits": 4, "gradeStatus": "pending" }
  ],
  "students": [
    { "id": 1, "name": "John Doe", "email": "john@example.com" }
  ],
  "enrollments": [
    { "id": 1, "studentId": 1, "courseId": 1 }
  ]
}

```

```

// src/app/interceptors/auth.interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authReq = req.clone({
    setHeaders: { Authorization: 'Bearer mock-token-12345' }
  });
  return next(authReq);
};

```



# Hands-On 9: State Management — NgRx Store, Actions, Reducers, Effects & Selectors

## Implementation Details

- **Redux Architecture:** Implemented NgRx Redux-pattern state architecture for state slice isolation.
- **Side Effects:** Handled asynchronous HTTP API side effects using NgRx Effects.

## Key Code Artifacts

```
// src/app/store/course/course.actions.ts
import { createAction, props } from '@ngrx/store';
import { Course } from '../../../models/course.model';

export const loadCourses = createAction('[Course] Load Courses');
export const loadCoursesSuccess = createAction('[Course] Load Courses Success', props<{ courses: Course[] }>());
export const loadCoursesFailure = createAction('[Course] Load Courses Failure', props<{ error: string }>());
```

```
// src/app/store/course/course.effects.ts
import { Injectable, inject } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { CourseService } from '../../../services/course.service';
import * as CourseActions from './course.actions';
import { catchError, map, switchMap, of } from 'rxjs';

@Injectable()
export class CourseEffects {
  private actions$ = inject(Actions);
  private courseService = inject(CourseService);

  loadCourses$ = createEffect(() =>
    this.actions$.pipe(
      ofType(CourseActions.loadCourses),
      switchMap(() =>
        this.courseService.getCoursesHttp().pipe(
          map(courses => CourseActions.loadCoursesSuccess({ courses })),
          catchError(error => of(CourseActions.loadCoursesFailure({ error: error.message })))
        )
      )
    )
  );
}
```

# Hands-On 10: Unit Testing Angular Applications — Jasmine, Karma & TestBed

## Implementation Details

- **Spec Testing:** Configured automated Jasmine spec tests verifying DOM assertions, event spy emitters, and MockStore provider integrations.

## Key Code Artifacts

```
// src/app/components/course-card/course-card.component.spec.ts
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { CourseCardComponent } from './course-card.component';
import { By } from '@angular/platform-browser';
```

```

describe('CourseCardComponent', () => {
  let component: CourseCardComponent;
  let fixture: ComponentFixture<CourseCardComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [CourseCardComponent]
    }).compileComponents();

    fixture = TestBed.createComponent(CourseCardComponent);
    component = fixture.componentInstance;
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should display course name in template', () => {
    component.course = { id: 1, name: 'Data Structures', code: 'CS101', credits: 4, gradeStatus: 'passed' };
    fixture.detectChanges();
    const titleEl = fixture.debugElement.query(By.css('h3')).nativeElement;
    expect(titleEl.textContent).toContain('Data Structures');
  });

  it('should emit enrollRequested on button click', () => {
    component.course = { id: 1, name: 'Data Structures', code: 'CS101', credits: 4, gradeStatus: 'passed' };
    fixture.detectChanges();
    spyOn(component.enrollRequested, 'emit');

    const button = fixture.debugElement.query(By.css('button')).nativeElement;
    button.click();

    expect(component.enrollRequested.emit).toHaveBeenCalledOnceWith(1);
  });
});

```

## Output :

Student Course Portal

[Home](#)
[Courses](#)
[Enroll](#)
[Profile](#)

## Welcome to the Student Course Portal

Manage your academic journey, browse courses, and monitor enrollment status easily.

COURSES AVAILABLE

12

ENROLLED

3

GPA

3.8

Searching for: **Angular****Angular Framework Essentials****PASSED**Code: **ANG101** | Credits: **3 Credits****Enroll**[Show Details](#)**Enterprise Architecture****PENDING**Code: **CS401** | Credits: **4 Credits****Unenroll**[Show Details](#)

## Student Enrollment Form

**Student Name \***

Name must be at least 3 characters.

**Student Email \***

Email is already registered (Async Validator Error).

**Course Code \***

Course code starting with 'XX' is restricted.

**Submit Application**